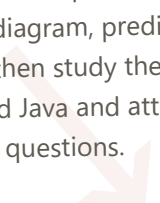


Built-in Packages and Imports

An import lets a source file use a simple type name; package declarations supply identity, and access modifiers supply visibility.

How to use this chapter:

Learn the idea in plain language, read each diagram, predict every trace row, then study the commented Java and attempt the exam-style questions.



The whole process at a glance

An import lets a source file use a simple type name; package declarations supply identity, and access modifiers supply visibility.

Think of a package as a class address. The package, filename, source root and compiled root have distinct jobs.

1

Identify the type

```
java.util.Scanner
```

java.util is the package; Scanner is the class. The fully qualified name already identifies it.

**2**

Choose a spelling

```
import java.util.Scanner;
```

A specific import lets this file write Scanner. java.util.* includes accessible types directly in java.util, not subpackages.

**3**

Check visibility

```
public class / public member
```

An import cannot grant access. A package-private member stays available only within its package.



Compile, then run

```
Scanner input = new Scanner(System.in);
```

The compiler resolves the simple name. The class must still be available; an import does not install or copy it.

The concept, plain language and vocabulary

A Java package supplies a namespace for types. An import declaration lets a compilation unit refer to an accessible type using its simple name instead of its fully qualified name.

Fully qualified
type `java.util.Scanner`



Import declaration
`import
java.util.Scanner;`



Simple spelling `Scanner`

Import changes which spelling is available in this source file; all three boxes refer to the same type.

TERM	MEANING IN THIS CHAPTER
Package	A namespace grouping related top-level types.
Import	A source-level name-shortening declaration.
Fully qualified name	Package plus type, such as <code>java.util.Scanner</code> .
Wildcard	Imports accessible types directly in that package; not its subpackages.
<code>java.lang</code>	Automatically imported; contains <code>String</code> , <code>System</code> and <code>Math</code> .

`java.util`
package

•

`Scanner`
class

The package prefix identifies the class; the import permits the shorter spelling `Scanner`.

An import does not install a library, execute code or copy a class. A dependency still has to be available to compilation and runtime.

PACKAGE	PURPOSE	EXAMPLES
java.lang	Core language types, automatic import	String, System, Math
java.util	Utilities and collections	Scanner, Arrays, ArrayList
java.io	File and stream I/O	File, BufferedReader, PrintWriter

Guided walkthrough: predict the next major step

Follow the class from source to execution

Identify the type

1

Identify the type

`java.util.Scanner`

java.util is the package; Scanner is the class. The fully qualified name already identifies it.

↓

java.util is the package; Scanner is the class. The fully qualified name already identifies it.

Choose a spelling

2

Choose a spelling

`import java.util.Scanner;`

A specific import lets this file write Scanner. java.util.* includes accessible types directly in java.util, not subpackages.

↓

A specific import lets this file write Scanner. java.util.* includes accessible types directly in java.util, not subpackages.

CSI247 · Built-in Packages and Imports

3 / 37

Check visibility

3

Check visibility

`public class / public member`

An import cannot grant access. A package-private member stays available only within its package.



An import cannot grant access. A package-private member stays available only within its package.

Compile, then run

4

Compile, then run

```
Scanner input = new Scanner(System.in);
```

The compiler resolves the simple name. The class must still be available; an import does not install or copy it.

The compiler resolves the simple name. The class must still be available; an import does not install or copy it.

▼ Full trace table - select a row to jump to its scene

STEP	STAGE	REPRESENTATION	REASON
1	Identify the type	<code>java.util.Scanner</code>	java.util is the package; Scanner is the class. The fully qualified name already identifies it.
2	Choose a spelling	<code>import java.util.Scanner;</code>	A specific import lets this file write Scanner. <code>java.util.*</code> includes accessible types directly in <code>java.util</code> , not subpackages.
3	Check visibility	<code>public class / public member</code>	An import cannot grant access. A package-private member stays available only within its package.
4	Compile, then run	<code>Scanner input = new Scanner(System.in);</code>	The compiler resolves the simple name. The class must still be available; an import does not install or copy it.

The trace at a glance

STAGE	REPRESENTATION	WHAT CHANGES
Identify the type	<code>java.util.Scanner</code>	<code>java.util</code> is the package; <code>Scanner</code> is the class. The fully qualified name already identifies it.
Choose a spelling	<code>import java.util.Scanner;</code>	A specific import lets this file write <code>Scanner</code> . <code>java.util.*</code> includes accessible types directly in <code>java.util</code> , not subpackages.
Check visibility	<code>public class / public member</code>	An import cannot grant access. A package-private member stays available only within its package.
Compile, then run	<code>Scanner input = new Scanner(System.in);</code>	The compiler resolves the simple name. The class must still be available; an import does not install or copy it.

Java packages

Group related classes together

A package is a named group of related classes organised in a matching folder structure. Packages improve maintainability and prevent class-name conflicts.

Built-in packages

Provided by Java, such as `java.lang`, `java.util`, and `java.io`.

User-defined packages

Created by programmers to organise their own related classes.

Common built-in examples

PACKAGE	PURPOSE	EXAMPLE
<code>java.lang</code>	Fundamental Java classes; imported automatically	<code>String</code> , <code>Math</code>
<code>java.util</code>	Utilities and collections	<code>Scanner</code> , <code>Random</code> , <code>Arrays</code>
<code>java.io</code>	File input and output	<code>PrintWriter</code> , <code>File</code>

A source file with no **package** statement belongs to the unnamed *default package*. Classes in a named package cannot import classes from the default package, so every file in this chapter declares a package. (The earlier array and file examples are single files in the default package, which is fine for a one-file program.)

The required folder hierarchy



The `pkg` folders under `src` and `classes` mirror each other. The package name is the folder path with dots instead of slashes.

Inside every packaged source file

A offers a method, B a second helper, and `Tester` is the class you run because it alone contains `main`.

src/pkg/A.java

```
package pkg; // first non-comment statement

public class A {
    // public: other packages may call this method after importing pkg.A
    public static void greet() {
        System.out.println("Hello from pkg.A");
    }
}
```

src/pkg/B.java

```
package pkg;

public class B {
    // public: visible everywhere
    public static int twice(int x) {
        return 2 * x;
    }

    // no modifier = package-private: visible only inside pkg
    static String secret() {
        return "pkg only";
    }
}
```

src/pkg/Tester.java

```
package pkg;

public class Tester {
    // The class you run must contain a main method like this one.
    public static void main(String[] args) {
        A.greet(); // same package: no import needed
        System.out.println(B.twice(21)); // prints 42
        System.out.println(B.secret()); // legal here: same package
    }
}
```

Imports and visibility

●●● Choose specific, wildcard, or fully qualified

```
import java.util.Scanner; // one class
import java.io.*;         // every public class directly in java.io
import pkg.A;              // a user-defined class

// Without an import you may write the fully qualified name instead:
pkg.A.greet();
```

- ☐ `java.lang` is imported automatically. Classes in the *same* package never need an import.
- ☐ A wildcard such as `java.io.*` imports the classes directly in that package, not classes in its sub-packages.
- ☐ An import only shortens names. It does not grant access to members that are not visible.

MODIFIER	SAME CLASS	SAME PACKAGE	SUBCLASS IN ANOTHER PACKAGE	ANY OTHER CLASS
public	Yes	Yes	Yes	Yes
protected	Yes	Yes	Yes	No
(none) · package-private	Yes	Yes	No	No
private	Yes	No	No	No

src/app/Main.java · uses another package

```
package app;

import pkg.A;    // one class from another package
import pkg.B;

public class Main {
    public static void main(String[] args) {
        A.greet();
        System.out.println(B.twice(5));    // prints 10
        // B.secret();    // compile error: secret() is not public
    }
}
```

Compile and run from the project folder

Commands

```
javac -d classes -sourcepath src src/pkg/*.java
java -classpath classes pkg.Tester

javac -d classes -sourcepath src src/app/Main.java
java -classpath classes app.Main
```

1

`-sourcepath src` tells the compiler where source files live.

2

`-d classes` tells it where to create the package folders and `.class` files.

3

`-classpath classes` tells Java where compiled packages begin.

4

`pkg.Tester` uses the fully qualified class name: package plus class. It must contain `main`.

5

Forward slashes in the paths also work on Windows command prompts.

Build a package from scratch

- 1 Create `src/pkg` and `classes`.
- 2 Place `package pkg;` first in each source file.
- 3 Save source files inside `src/pkg`.
- 4 Give one class a `main` method.
- 5 Compile with `-d classes`.
- 6 Run the fully qualified class name.

Debug checklist

- Package name and folder path must match.
- Java is case-sensitive.
- Run from the folder above `classes`.
- Do not include `.class` in the run command.
- The class you run needs `main`.

Reading the error messages

MESSAGE	USUAL CAUSE	FIX
Error: Could not find or load main class <code>pkg.Tester</code>	Running from the wrong folder, a wrong classpath, a misspelled name, or adding <code>.class</code> .	Run from the folder that contains <code>classes</code> ; use <code>-classpath classes pkg.Tester</code> with no extension.
Error: Main method not found in class <code>pkg.A</code>	The class has no <code>public static void main(String[] args)</code> .	Run a class that has <code>main</code> , such as <code>pkg.Tester</code> .
error: package <code>pkg</code> does not exist	Compiling <code>app.Main</code> without telling <code>javac</code> where <code>pkg</code> lives.	Add <code>-sourcepath src</code> (or <code>-classpath classes</code> if <code>pkg</code> is already compiled).
error: class <code>Wrong</code> is public, should be declared in a file named <code>Wrong.java</code>	A public class name must match its file name.	Rename the file or the class.
error: <code>secret()</code> is not public in <code>B</code> ; cannot be accessed from outside package	Calling a package-private member from another package.	Make it <code>public</code> , or call it only from inside <code>pkg</code> .
Compiles, but <code>java pkg.A</code> says it cannot find the class	The <code>package</code> statement does not match the folder, so the <code>.class</code> landed in a different folder.	Make the <code>package</code> statement, the source folder, and the run command agree.

Worked example · source to running class

STAGE	INPUT	WHAT JAVA RESOLVES	RESULT
Declare	<code>package pkg; in src/pkg/Tester.java</code>	The source folder path must equal the package name	<code>src/pkg/Tester.java</code>
Compile	<code>javac -d classes - sourcepath src src/pkg/*.java</code>	<code>-d classes</code> creates <code>classes/pkg/</code> ; one <code>.class</code> per class	<code>classes/pkg/A.class, B.class, Tester.class</code>
Run	<code>java -classpath classes pkg.Tester</code>	Classpath root <code>classes</code> + <code>pkg.Tester</code> → <code>classes/pkg/Tester.class</code> ; then <code>main</code> runs	Hello from pkg.A · 42 · pkg only
Cross-package	<code>import pkg.A; in src/app/Main.java</code>	<code>-sourcepath src</code> lets <code>javac</code> find <code>src/pkg/A.java</code> when compiling <code>app.Main</code>	<code>classes/app/Main.class</code> ; prints Hello from pkg.A · 10

 Console output of `java -classpath classes pkg.Tester`

```
Hello from pkg.A
42
pkg only
```

Fully annotated Java and commands

java

```
package app; // The package declaration precedes imports.

import java.util.Scanner; // Makes the simple name Scanner available.
import java.util.Arrays; // Imports one public type, not a whole library.

public class ImportDemo {
    public static void main(String[] args) {
        // System and String come from automatically imported java.lang.
        Scanner input = new Scanner(System.in);
        System.out.print("Enter one whole number: ");
        int value = input.nextInt(); // This example expects valid integer input.
        int[] values = {value, 2, 1};
        Arrays.sort(values); // Static utility method sorts in ascending order.
        System.out.println(Arrays.toString(values));
        // Equivalent spelling without a Scanner import:
        // java.util.Scanner input = new java.util.Scanner(System.in);
    }
}
```

Input 5 prints [1, 2, 5] after sorting. Save at src/app/ImportDemo.java.

Commands

```
javac -d classes src/app/ImportDemo.java
java -classpath classes app.ImportDemo
```

Complete companion project for the package workflow

The following four source files make the earlier diagram and the later output/practice tables reproducible within this guide. Their package declarations and paths are explicit.

java

```
package pkg; // Identity pkg.A; file src/pkg/A.java.

public class A { // Public class name must match A.java.
    // Other packages may call this public method on this public class.
    public static void greet() {
        System.out.println("Hello from pkg.A");
    }
}
```

java

```
package pkg; // File src/pkg/B.java.

public class B {
    public static int twice(int x) { return 2 * x; }
    // No access modifier: only classes in package pkg may call secret.
    static String secret() { return "pkg only"; }
}
```

java

```
package pkg; // File src/pkg/Tester.java.

public class Tester {
    // This exact main signature supplies the program entry point.
    public static void main(String[] args) {
        A.greet(); // Same package: no import required.
        System.out.println(B.twice(21)); // Prints 42.
        System.out.println(B.secret()); // Legal: Tester and B share pkg.
    }
}
```

java

```
package app; // File src/app/Main.java.

import pkg.A; // Import a public class from another package.
import pkg.B;

public class Main {
    public static void main(String[] args) {
        A.greet();
        System.out.println(B.twice(5)); // Prints 10.
        // B.secret(); // Would fail: imports do not grant package-private access.
    }
}
```

Commands

```
# Run from the project folder that contains src and classes.
javac -d classes -sourcepath src src/pkg/A.java src/pkg/B.java src/pkg/Tester.java
src/app/Main.java

# Classpath points to the ROOT above pkg and app, not to classes/pkg.
java -classpath classes pkg.Tester
java -classpath classes app.Main
```

Imports and access: different responsibilities

MEMBER MODIFIER	SAME CLASS	SAME PACKAGE	SUBCLASS OUTSIDE PACKAGE	UNRELATED OUTSIDE CLASS
public	Yes	Yes	Yes	Yes
protected	Yes	Yes	Yes, with subclass access restrictions	No
none (package-private)	Yes	Yes	No	No
private	Yes	No	No	No

Protected access from a different package is through inheritance and is subject to receiver restrictions; it is not blanket permission to access any superclass object. A top-level class may be public or package-private, not protected/private.

Access comparison

```
import pkg.B;
// Legal in app.Main: B.twice(5)
// Illegal in app.Main: B.secret()
// Importing B does not alter secret() visibility.
```

Properties and practical cost

Packages organize names and collaboration; imports are compile-time name resolution conveniences. A wildcard is not a performance optimization. There is no useful sorting-style $O(n)$ claim to assign to an import declaration.

PROPERTY	PRACTICAL CONSEQUENCE
Namespace	Same simple type names may exist in different packages.
Encapsulation	Package-private members form an internal package API.
Maintainability	Related classes have predictable source locations.
Dependency lookup	Sourcepath/classpath roots must locate the required types.
Case sensitivity	pkg.A and pkg.a name different types.

Five worked package cases

1. Specific import

Given

```
import java.util.Scanner;
```

→

Resolution / result

```
Scanner in = new Scanner(System.in);
```

The single imported type is available by its simple name.

STEP	EVIDENCE	CONCLUSION
1	<code>import java.util.Scanner;</code>	Identify package/type/root
2	<code>Scanner in = new Scanner(System.in);</code>	The single imported type is available by its simple name.

2. Fully qualified alternative

GivenNo Scanner import

→

Resolution / result`java.util.Scanner in = new java.util.Scanner(System.in);`

Same type; longer spelling resolves its identity directly.

STEP	EVIDENCE	CONCLUSION
1	No Scanner import	Identify package/type/root
2	<code>java.util.Scanner in = new java.util.Scanner(System.in);</code>	Same type; longer spelling resolves its identity directly.

3. Wildcard boundary

Given`import java.util.*;`

→

Resolution / result`Scanner` works; a subpackage class still needs its own import.

Wildcard imports do not recurse into subpackages.

STEP	EVIDENCE	CONCLUSION
1	<code>import java.util.*;</code>	Identify package/type/root
2	<code>Scanner</code> works; a subpackage class still needs its own import.	Wildcard imports do not recurse into subpackages.

4. Automatic names

GivenNo `import java.lang.String;`

→

Resolution / resultString `message = "Hello";`

java.lang types are automatically imported.

STEP	EVIDENCE	CONCLUSION
1	No <code>import java.lang.String;</code>	Identify package/type/root
2	<code>String message = "Hello";</code>	java.lang types are automatically imported.

5. Ambiguous simple name

Given`import java.util.*; import java.sql.*;`

→

Resolution / result`java.util.Date chosen = new java.util.Date();`

Both packages define Date; qualify the required type.

STEP	EVIDENCE	CONCLUSION
1	<code>import java.util.*; import java.sql.*;</code>	Identify package/type/root
2	<code>java.util.Date chosen = new java.util.Date();</code>	Both packages define Date; qualify the required type.

Common errors and exact repairs

MESSAGE / SYMPTOM	LIKELY CAUSE	REPAIR
cannot find symbol: Scanner	Simple name is not in scope.	Add <code>import java.util.Scanner</code> or use the fully qualified name.

MESSAGE / SYMPTOM	LIKELY CAUSE	REPAIR
reference to Date is ambiguous	Two wildcard-imported packages contain Date.	Use a specific import or fully qualified name.
package pkg does not exist	Compiler cannot locate sources/compiled dependencies.	Use -sourcepath src or -classpath classes as appropriate.
class A is public, should be declared in A.java	Public class and filename disagree.	Use A.java with public class A.
Could not find or load main class pkg.Tester	Wrong classpath root/name or missing compiled file.	Use classes as root and pkg.Tester without .class.
Main method not found in class pkg.A	That helper is not an entry-point class.	Run pkg.Tester or app.Main.
secret() is not public in B	Cross-package access to a package-private member.	Use a public API; an import cannot grant visibility.

Exam-style practice with worked answers

▼ A. Place Helper in ub.cs.tools. Give declaration, source path, output path and binary name. [4]

package ub.cs.tools; / src/ub/cs/tools/Helper.java / classes/ub/cs/tools/Helper.class / ub.cs.tools.Helper.
The src and classes roots are not package segments.

▼ B. Explain import java.util.* and whether it includes subpackages. [3]

It allows accessible types directly in java.util to be named simply. It does not import types from subpackages, copy code or install dependencies.

▼ **C. Why can `pkg.Tester` call `B.secret` while `app.Main` cannot? [3]**

`secret` has no access modifier, so it is package-private. `Tester` shares `pkg`. `Main` belongs to `app`; importing `B` does not grant member visibility.

▼ **D. Repair `java -classpath classes/pkg Tester.class`. [3]**

Use `java -classpath classes pkg.Tester`. The classpath names the root above package folders; the launch argument is a dotted binary name without an extension.

▼ **E. Compile and run the four classes in this chapter. State expected output. [6]**

Commands

```
# Run from the project folder that contains src and classes.
javac -d classes -sourcepath src src/pkg/A.java src/pkg/B.java
src/pkg/Tester.java src/app/Main.java

# Classpath points to the ROOT above pkg and app, not to classes/pkg.
java -classpath classes pkg.Tester
java -classpath classes app.Main
```

`pkg.Tester` prints Hello from `pkg.A`, 42, `pkg` only. `app.Main` prints Hello from `pkg.A`, 10.

▼ **F. `Scanner` is unresolved even though the JDK is installed. What are two fixes? [2]**

Add `import java.util.Scanner`; or use `java.util.Scanner` at every relevant type/constructor use. Both name the same class.

An array is one fixed-size row of same-type values

An array stores a numbered sequence. Its length is fixed when it is created, its first index is 0, and its final valid index is always `length - 1`.

63	29	72	85	18
INDEX 0	INDEX 1	INDEX 2	INDEX 3	INDEX 4

TYPE

int[]

LENGTH

5

FIRST INDEX

0

LAST INDEX

4

Declare, create, initialise, use

● ● ● Array basics (code fragment)

```
// 1. Declare a variable that can refer to an int array.
int[] marks;

// 2. Create five int cells. Every cell starts at 0.
marks = new int[5];

// 3. Store values by index.
marks[0] = 63;
marks[marks.length - 1] = 18; // last valid cell

// A literal declares, creates, and fills in one statement.
int[] ready = {63, 29, 72, 85, 18};

// Reading index 5 would throw ArrayIndexOutOfBoundsException.
System.out.println(ready[2]); // prints 72
```

Fixed size does not mean fixed values. You may replace values in existing cells, but you cannot make the same array gain a sixth cell. Create a new array or use an **ArrayList** when size must grow.

▼ Quick check: why is `marks[marks.length]` invalid?

For length 5, valid indexes are 0 through 4. `marks.length` is 5, one position beyond the array.

Array helpers you will meet

These come from `java.util.Arrays` (add `import java.util.Arrays;`). The file and merge-sort examples later in this guide use them.

CALL	WHAT IT DOES	RESULT FOR THE EXAMPLE
<code>Arrays.toString(a)</code>	Readable text for the contents. Printing an array directly shows an unreadable reference such as <code>[I@6d06d69c</code> .	<code>[63, 29, 72, 85, 18]</code>
<code>Arrays.copyOf(a, n)</code>	New array of length <code>n</code> . Copies the first <code>n</code> values; extra cells get default values.	<code>int[] a = {63, 29, 72};</code> then <code>Arrays.copyOf(a, 5) → [63, 29, 72, 0, 0]</code>
<code>Arrays.copyOfRange(a, from, to)</code>	New array from index from up to, but not including , <code>to</code> .	<code>int[] a = {63, 29, 72, 85};</code> then <code>Arrays.copyOfRange(a, 1, 3) → [29, 72]</code>
<code>Arrays.fill(a, v)</code>	Stores <code>v</code> in every cell.	<code>int[] a = new int[4]; Arrays.fill(a, 7);</code> → <code>a</code> is <code>[7, 7, 7, 7]</code>
<code>Arrays.equals(a, b)</code>	Compares contents cell by cell. Both <code>a == b</code> and <code>a.equals(b)</code> only ask whether the two variables reach the <i>same</i> array.	Two separate <code>{1,2,3}</code> arrays: <code>== false</code> , <code>.equals false</code> , <code>Arrays.equals true</code>

Length syntax differs. An array uses `marks.length` (no parentheses). A String uses `name.length()`.

▼ Quick check: what does `Arrays.copyOfRange(a, 1, 3)` return for `a = {63, 29, 72, 85}`?

`[29, 72]`. It copies indexes 1 and 2. The end index 3 is excluded, which is why merge sort writes `m + 1` for the left half.

Applications / File → array → sort

Read a file, fill an array, then sort only the values read

A file supplies text tokens, a Scanner parses integers, an array stores them, and the insertion sort from the sorting chapter orders the used portion. The program must track how many cells are really in use.

1

File text

63 29 bad 72 85 18

2

Parse

Accept integers; report and skip bad.

3

Store

[63, 29, 72, 85, 18] with `count = 5`.

4

Sort

[18, 29, 63, 72, 85].

Capacity versus logical size

`buffer.length` is the **capacity**: 100 cells always exist. `count` is the **logical size**: only cells `0 .. count - 1` hold file values. The first eight of the 100 cells are drawn here; the rest also hold 0.



```
import java.io.File;
import java.io.FileNotFoundException;
import java.util.Arrays;
import java.util.Scanner;

public class FileArraySort {
    public static void main(String[] args) throws FileNotFoundException {
        int[] marks = readIntegers("marks.txt");
        System.out.println("Read: " + Arrays.toString(marks));
        insertionSort(marks); // apply the course algorithm
        System.out.println("Sorted: " + Arrays.toString(marks));
    }

    public static int[] readIntegers(String filename)
        throws FileNotFoundException {
        int[] buffer = new int[100]; // maximum capacity
        int count = 0;               // values actually stored

        // try-with-resources closes the Scanner automatically.
        try (Scanner input = new Scanner(new File(filename))) {
            while (input.hasNext() && count < buffer.length) {
                if (input.hasNextInt()) {
                    buffer[count] = input.nextInt();
                    count++; // advance only after storing a valid int
                } else {
                    System.out.println("Skipped token: " + input.next());
                }
            }
        }
        // Remove unused zero-filled cells before sorting.
        return Arrays.copyOf(buffer, count);
    }

    public static void insertionSort(int[] a) {
        for (int i = 1; i < a.length; i++) {
            int key = a[i];
            int j = i - 1;
            while (j ≥ 0 && key < a[j]) {
                a[j + 1] = a[j];
                j--;
            }
            a[j + 1] = key;
        }
    }
}
```

With `marks.txt` containing `63 29 bad 72 85 18`, the program prints:

Console output

```
Skipped token: bad
Read:  [63, 29, 72, 85, 18]
Sorted: [18, 29, 63, 72, 85]
```

Basic trace · parsing marks.txt

TOKEN	INTEGER?	COUNT BEFORE	ACTION	USED VALUES AFTER
63	Yes	0	Store at index 0; count++	[63]
29	Yes	1	Store at index 1; count++	[63, 29]
bad	No	2	Consume and skip; count unchanged	[63, 29]
72	Yes	2	Store at index 2; count++	[63, 29, 72]
85	Yes	3	Store at index 3; count++	[63, 29, 72, 85]
18	Yes	4	Store at index 4; count++	[63, 29, 72, 85, 18]

Before sorting

63	29	72	85	18
0	1	2	3	4

After insertion sort

18	29	63	72	85
0	1	2	3	4

If the file is missing. new File("marks.txt") is searched for in the program's working directory: the folder you ran java from, or the project folder in many IDEs. If it is not there, new Scanner(...) throws FileNotFoundException(marks.txt (No such file or directory) on Linux and macOS; Windows words it (The system cannot find the file specified)) before the loop starts. That checked exception is why main and readIntegers say throws FileNotFoundException. An *empty* file is not an error: count stays 0 and the result is [].

Application questions

▼ Why keep count separately from buffer.length?

buffer.length is capacity: 100 cells exist. count is logical size: only the first count cells contain values read

from the file. Sorting all 100 would drag unused zeros into the result.

▼ Why must the invalid token be consumed with `input.next()`?

`hasNextInt()` only looks at the next token; it does not consume it. Without `input.next()` the Scanner stays on the same invalid token and the loop never progresses.

▼ What should happen if the file contains more than 100 integers?

This version stops when the buffer is full and ignores the rest, because the loop condition includes `count < buffer.length`. To keep every value, remove that part of the condition and grow the array inside the `if (input.hasNextInt())` branch (below), or use an `ArrayList<Integer>`.

● ● ● Doubling the capacity

```
// Change the loop condition: drop "&& count < buffer.length" so the loop keeps reading.
while (input.hasNext()) {
    if (input.hasNextInt()) {
        // Grow instead of stopping: double the capacity when the buffer is full.
        if (count == buffer.length) {
            buffer = Arrays.copyOf(buffer, buffer.length * 2);
        }
        buffer[count] = input.nextInt();
        count++;
    } else {
        System.out.println("Skipped token: " + input.next());
    }
}
```

▼ Why does `return Arrays.copyOf(buffer, count)` help the sort?

It returns an array whose `length` equals the number of real values. `insertionSort` loops to `a.length`, so it never touches the unused zero cells.

▼ Practice: trace the file 40 x 15 15 9. What is printed?

TOKEN	INTEGER?	COUNT BEFORE	ACTION	USED VALUES AFTER
40	Yes	0	Store at index 0; count++	[40]
x	No	1	Consume and skip	[40]
15	Yes	1	Store at index 1; count++	[40, 15]
15	Yes	2	Store at index 2; count++ (duplicates are kept)	[40, 15, 15]
9	Yes	3	Store at index 3; count++	[40, 15, 15, 9]

The program prints **Skipped token: x**, then **Read: [40, 15, 15, 9]**, then **Sorted: [9, 15, 15, 40]**. **count** ends at 4.

Arrays and file input in one sentence

Indexes connect loops to array cells; primitive cells store values, reference cells store nullable object references, and file input needs both capacity control and a logical count before sorting.

Applications / Objects by key field

Sort and search an array of objects by one field

Binary search needs the array sorted “by the same key” it searches. For objects the key is a field you choose. The algorithms are unchanged: the comparison reads the field through a getter instead of comparing the cell directly.

```
public class Staff {  
    private String name;  
    private int staffId;  
    private String department;  
  
    public Staff(String name, int staffId, String department) {  
        this.name = name;  
        this.staffId = staffId;  
        this.department = department;  
    }  
  
    public String getName() { return name; }  
    public int getId() { return staffId; }  
    public void setId(int staffId) { this.staffId = staffId; }  
    public void setDepartment(String department) { this.department = department; }  
}
```

```
public class StaffTools {
    // Insertion sort by id over the USED part only: staff[0 .. count - 1].
    public static void sortById(Staff[] staff, int count) {
        for (int i = 1; i < count; i++) {
            Staff key = staff[i];           // save the object reference
            int j = i - 1;
            while (j ≥ 0 && key.getId() < staff[j].getId()) {
                staff[j + 1] = staff[j];    // shift the reference right
                j--;
            }
            staff[j + 1] = key;
        }
    }

    // Binary search by id. The array must already be sorted by id.
    public static int findById(Staff[] staff, int count, int id) {
        int low = 0, high = count - 1;
        while (low ≤ high) {
            int mid = low + (high - low) / 2;
            int midId = staff[mid].getId();
            if (midId == id) return mid;
            if (midId < id) low = mid + 1;
            else high = mid - 1;
        }
        return -1;
    }

    public static void main(String[] args) {
        Staff[] staff = new Staff[6];      // capacity 6
        staff[0] = new Staff("Mike", 17, "Bio");
        staff[1] = new Staff("Ana", 3, "Mat");
        staff[2] = new Staff("Raj", 11, "Phy");
        staff[3] = new Staff("Lee", 9, "Bio");
        int count = 4;                     // only four cells are used

        sortById(staff, count);
        for (int i = 0; i < count; i++) {
            System.out.println(staff[i].getId() + " " + staff[i].getName());
        }
        System.out.println("id 11 is at index " + findById(staff, count, 11));
        System.out.println("id 99 is at index " + findById(staff, count, 99));
    }
}
```

Each cell holds a *reference*. Shifting `staff[j + 1] = staff[j]` moves references, never copies Staff objects. The ids and names are used here, with names shown beneath each cell:

Before

17	3	11	9
MIKE	ANA	RAJ	LEE

After sorting by id

3	9	11	17
ANA	LEE	RAJ	MIKE

Insertion-sort trace by id

KEY (ID)	COMPARISONS	SHIFTS	ARRAY OF IDS AFTER INSERTION
3	1	1	[3, 17, 11, 9]
11	2	1	[3, 11, 17, 9]
9	3	2	[3, 9, 11, 17]
Total	6	4	Sorted by id

Binary search for id 11

STEP	LOW	HIGH	MID	ID AT MID	DECISION
1	0	3	1	9	9 < 11 → low = 2
2	2	3	2	11	Match → return 2

`findById(staff, 4, 11)` returns 2 after 2 comparisons. For id 99 the range empties and it returns -1.

The used part versus the whole array

The staff array has capacity 6 but only 4 cells are filled. Cells 4 and 5 are null.



That is why both methods take `count`. If `sortById` were called with `staff.length`, the loop would reach `staff[4]`, and `key.getId()` would throw `NullPointerException` because the cell holds no `Staff` object.

Sorting by text instead of a number

Use `compareTo`, never `<`, for Strings. It returns a negative number if the left String sorts first, zero if equal, and a positive number otherwise. Only the while line changes:

● ● ● Sort by name

```
// Text key: compareTo returns a negative number when the left String sorts first.
while (j ≥ 0 && key.getName().compareTo(staff[j].getName()) < 0) {
```

The same `main` then prints `3 Ana, 9 Lee, 17 Mike, 11 Raj`: alphabetical by name, so the ids are now out of order. Its two lookups therefore print `id 11 is at index -1` and `id 99 is at index -1`. Binary search by id fails on this array, even though id 11 is present. Binary search by name would need the matching `compareTo` test as well, on an array sorted by name.

▼ Quick check: why would binary search by id fail on an array sorted by name?

The array is ordered by name, so ids appear in any order. Comparing the id at the midpoint says nothing about which half holds the target, and the search can discard the half that contains it.

▼ Quick check: after `staff[0] = staff[1];`, how many `Staff` objects exist?

The number of objects is unchanged. The statement copies a reference, so `staff[0]` and `staff[1]` now reach the same object, and the object previously at index 0 may become unreachable.

Past paper / Q2 · 15 marks · Read a file into Book objects

Read a file into Book objects

Follow one record all the way from text, through parsing, to a non-null array cell.

Questions 2(a)–(c): imports, reader, and array

APPROACH	REQUIRED IMPORTS	OPEN THE CURRENT-DIRECTORY FILE
The question's <code>FileReader</code> route	<code>java.io.FileReader</code> , <code>BufferedReader</code> , <code>IOException</code>	<code>FileReader fr = new FileReader("books.txt");</code>
Scanner alternative	<code>java.io.File</code> , <code>FileNotFoundException</code> , <code>java.util.Scanner</code>	<code>new Scanner(new File("books.txt"))</code>

`new Book[50]` creates one array with capacity 50. Every cell starts as `null`; it does not call the `Book` constructor 50 times.

A 50-cell reference array after reading three records

B1	B2	B3	null	null	null
0	1	2	3	4	5

Shown: cells 0–5. Cells 6–49 are also null. count = 3.

Question 2(d): turn each file line into one object

books.txt used for the trace

Hobbit, 20.99
Dune, 15.50
Emma, 9.75

LINE	AFTER SPLIT AND TRIM	BOOK STORED	COUNT AFTER
Hobbit, 20.99	title = Hobbit; price = 20.99	books[0]	1
Dune, 15.50	title = Dune; price = 15.50	books[1]	2
Emma, 9.75	title = Emma; price = 9.75	books[2]	3

Book.java

```
public class Book {
    private String title;
    private double price;

    public Book(String ttl, double pr) {
        title = ttl;
        price = pr;
    }
    public String getTitle() { return title; }
    public double getPrice() { return price; }
    public void setTitle(String ttl) { title = ttl; }
    public void setPrice(double pr) { price = pr; }
    public String toString() { return title + " (" + price + ")"; }
    public boolean equals(Book bk) { return title.equals(bk.title) && price == bk.price; }
}
```

```

import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;

public class BooksApp {
    public static void main(String[] args) {
        Book[] books = new Book[50];           // 50 empty (null) cells
        int count = 0;                          // how many cells really hold a Book

        try (BufferedReader br = new BufferedReader(new FileReader("books.txt"))) {
            String line;
            while ((line = br.readLine()) != null && count < books.length) {
                String[] details = line.split(",");           // "Hobbit,20.99" → ["Hobbit", "20.99"]
                String title = details[0].trim();
                double price = Double.parseDouble(details[1].trim());
                books[count] = new Book(title, price);
                count++;
            }
        } catch (IOException e) {
            System.out.println("Error reading file: " + e.getMessage());
        }

        for (int i = 0; i < count; i++) {           // only the filled cells
            System.out.println(books[i].getTitle());
        }
    }
}

```

The checked exception is handled by `catch (IOException e)`; try-with-resources closes the reader even if parsing fails. `trim()` removes spaces around comma-separated fields, and `Double.parseDouble` converts the price text.

Question 2(e): print only cells that contain objects

The 50-cell trap. If only three records were read, an enhanced-for loop over all 50 cells eventually calls `getTitle()` on `null` and throws `NullPointerException`. Loop from 0 to `count - 1`. Use `println` because the question requires one title per line.

```
import java.io.File;
import java.io.FileNotFoundException;
import java.util.Scanner;

public class BooksScanner {
    public static void main(String[] args) {
        Book[] books = new Book[50];
        int count = 0;

        try (Scanner in = new Scanner(new File("books.txt"))) {
            while (in.hasNextLine() && count < books.length) {
                String[] details = in.nextLine().split(",");
                books[count] = new Book(details[0].trim(), Double.parseDouble(details[1].trim()));
                count++;
            }
        } catch (FileNotFoundException e) {
            System.out.println("File not found!");
        }

        for (int i = 0; i < count; i++) {
            System.out.println(books[i].getTitle());
        }
    }
}
```

▼ Why keep count separate from books.length?

books.length is capacity: always 50. count is logical size: how many Book objects were actually stored. Algorithms that read the data must stop at count.

Topic quiz: test your understanding

Answer all 12 questions. Choose an option to see the reasoning; reset to retry. The printable answer key follows the questions.

1. What does an import do?

- ☐ A. Allows an accessible type to be written by a shorter name
- ☐ B. Installs a library
- ☐ C. Changes private members to public

2. Which package is automatically imported?

- ☐ A. java.util
- ☐ B. java.io
- ☐ C. java.lang

3. Does `import java.util.*` include `java.util.concurrent` types?

- ☐ A. Only if they are public
- ☐ B. No
- ☐ C. Yes, recursively

4. What comes first in a packaged source file, after comments?

- ☐ A. package declaration
- ☐ B. import declarations
- ☐ C. public class declaration

5. What is the identity of public class `Helper` declared in `ub.cs.tools`?

- ☐ A. `src.ub.cs.tools.Helper`
- ☐ B. `Helper.java`
- ☐ C. `ub.cs.tools.Helper`

6. Correct source path below source root `src`?

- ☐ A. `ub/cs/src/Helper.java`
- ☐ B. `src/ub/cs/tools/Helper.java`
- ☐ C. `src/ub.cs.tools/Helper.class`

7. What does `javac -d` classes select?

- ☐ A. The output root for compiled classes
- ☐ B. The package name
- ☐ C. The class containing main

8. How do you launch `classes/pkg/Tester.class` from the project root?

- ☐ A. `java -classpath classes/pkg Tester.class`
- ☐ B. `javac pkg.Tester`
- ☐ C. `java -classpath classes pkg.Tester`

9. Can `app.Main` use a package-private method in `pkg.B` after importing `B`?

- ☐ A. Yes, if it is static
- ☐ B. No
- ☐ C. Yes, imports grant access

10. Must two classes in the same package import one another?

- ☐ A. No
- ☐ B. Yes, always
- ☐ C. Only if both are public

11. How can a conflict between `java.util.Date` and `java.sql.Date` be resolved?

- ☐ A. Add more wildcard imports
- ☐ B. Rename `java.lang`
- ☐ C. Use a fully qualified name or the appropriate specific import

12. What does a wrong classpath root usually prevent?

- ☐ A. Allocating an array
- ☐ B. Finding the compiled main class
- ☐ C. Parsing the package declaration in source

▼ Answer key and explanations

1. **Allows an accessible type to be written by a shorter name.** Imports affect compile-time name resolution, not installation or access control.
2. **java.lang.** String, System and Math are java.lang types and can use simple names automatically.
3. **No.** A wildcard import covers accessible types directly in that package, not its subpackages.
4. **package declaration.** The normal source order is package, imports, then top-level type declarations.
5. **ub.cs.tools.Helper.** Source roots and filename extensions are not part of the fully qualified class name.
6. **src/ub/cs/tools/Helper.java.** Package segments map to directories below the root; source uses the .java extension.
7. **The output root for compiled classes.** The compiler creates package directories beneath the output root.
8. **java -classpath classes pkg.Tester.** The classpath ends above pkg; the launch argument is a dotted name with no extension.
9. **No.** Importing a class never widens its members' access.
10. **No.** Accessible types in the same package can be named directly.
11. **Use a fully qualified name or the appropriate specific import.** Qualification tells the compiler exactly which Date you mean.
12. **Finding the compiled main class.** Runtime lookup combines the root with the dotted binary name to locate the class file.